

JAVA WITH DSA-DAY 10

GIVEN THE PUSH AND POP SEQUENCES OF A STACK, DETERMINE WHETHER THE POP SEQUENCE IS VALID (STACK PERMUTATION)

```
import java.util.*;

class Main {

    public static void main(String[] args) {

        int n=5;

        int pushed[]={1,2,3,4,5};

        int popped[]={4,5,3,2,1};

        int index=0;

        Stack<Integer> stack=new Stack<>();

        for(int i=0;i<n;i++){

            stack.push(pushed[i]);

            if(stack.peek()==popped[index]){

                stack.pop();

                index++;

            }

        }

        for(int i=index;i<n;i++){

            if(stack.peek()==popped[index]){

                stack.pop();

                index++;

            }

        }

        if(stack.isEmpty()){

            System.out.println("true");

        }

        else{

            System.out.println("false");

        }

    }

}
```

OUTPUT:

True

Using while:

```

import java.util.*;
class Main {
    public static void main(String[] args) {
        int n=5;
        int pushed[]={1,2,3,4,5};
        int popped[]={4,5,3,2,1};
        int index=0;
        Stack<Integer> stack=new Stack<>();
        for(int i=0;i<n;i++){
            stack.push(pushed[i]);
            while(!stack.isEmpty() && stack.peek()==popped[index]){
                stack.pop();
                index++;
            }
        }
        if(stack.isEmpty()){
            System.out.println("true");
        }
        else{
            System.out.println("false");
        }
    }
}

```

OUTPUT:

True

CHECK WHETHER A GIVEN INPUT (NUMBERS OR TEXT) IS A PALINDROME, WHILE IGNORING SPACES AND SPECIAL CHARACTERS

```

import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter String or Number: ");
        String input = sc.nextLine();
        // Remove spaces and special characters, convert to lowercase
        input = input.replaceAll("[^a-zA-Z0-9]", "").toLowerCase();
    }
}

```

```

Stack<Character> stack = new Stack<>();
// Push all characters into stack
for (int i = 0; i < input.length(); i++) {
    stack.push(input.charAt(i));
}
String reversed = "";
// Pop characters to create reversed string
while (!stack.isEmpty()) {
    reversed += stack.pop();
}
// Compare
if (input.equals(reversed)) {
    System.out.println("Palindrome");
} else {
    System.out.println("Not Palindrome");
}
}
}

```

OUTPUT:

Enter String or Number: A man, a plan, a canal: Panama
 Palindrome

Using string builder:

```

import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter String or Number: ");
        String input = sc.nextLine();
        StringBuilder clean = new StringBuilder();
        // Keep only letters and digits
        for (int i = 0; i < input.length(); i++) {
            char ch = input.charAt(i);
            if (Character.isLetter(ch) || Character.isDigit(ch)) {
                clean.append(Character.toLowerCase(ch));
            }
        }
    }
}

```

```

    }
}
Stack<Character> stack = new Stack<>();
// Push characters into stack
for (int i = 0; i < clean.length(); i++) {
    stack.push(clean.charAt(i));
}
StringBuilder reverse = new StringBuilder();
// Pop to create reversed string
while (!stack.isEmpty()) {
    reverse.append(stack.pop());
}
// Compare
if (clean.toString().equals(reverse.toString())) {
    System.out.println("Palindrome");
} else {
    System.out.println("Not Palindrome");
}
}
}

```

OUTPUT:

Enter String or Number: Mad::: am
 Palindrome

QUEUE

OPERATIONS:

```

import java.util.*;

public class Main {

    public static void main(String[] args) {

        Queue<Integer> q = new LinkedList<>();

        // Enqueue
        q.add(10);
        q.offer(20);
        q.add(30);
        q.offer(40);
    }
}

```

```
System.out.println("Queue: " + q);
// Peek
System.out.println("Front using peek(): " + q.peek());
// Element
System.out.println("Front using element(): " + q.element());
// Contains
System.out.println("Contains 20? " + q.contains(20));
// Size
System.out.println("Size: " + q.size());
// Iterator traversal
System.out.print("Using Iterator: ");
Iterator<Integer> it = q.iterator();
while (it.hasNext()) {
    System.out.print(it.next() + " ");
}
System.out.println();
// Convert to Array
Object arr[] = q.toArray();
System.out.print("Queue to Array: ");
for (Object x : arr) {
    System.out.print(x + " ");
}
System.out.println();
// Dequeue using poll
System.out.println("Removed using poll(): " + q.poll());
// Dequeue using remove
System.out.println("Removed using remove(): " + q.remove());
System.out.println("Queue after deletion: " + q);
// Empty check
System.out.println("Is Empty? " + q.isEmpty());
// Clear queue
q.clear();
System.out.println("After clear(): " + q);
System.out.println("Is Empty Now? " + q.isEmpty());
```

```
}  
}
```

OUTPUT:

Queue: [10, 20, 30, 40]

Front using peek(): 10

Front using element(): 10

Contains 20? true

Size: 4

Using Iterator: 10 20 30 40

Queue to Array: 10 20 30 40

Removed using poll(): 10

Removed using remove(): 20

Queue after deletion: [30, 40]

Is Empty? false

After clear(): []

Is Empty Now? True

ARRAYDEQUE

```
import java.util.*;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        ArrayDeque<Integer> dq = new ArrayDeque<>();
```

```
        // INSERT OPERATIONS
```

```
        dq.add(10);
```

```
        dq.addFirst(5);
```

```
        dq.addLast(20);
```

```
        dq.offer(30);
```

```
        dq.offerFirst(1);
```

```
        dq.offerLast(40);
```

```
        System.out.println("After Insertions: " + dq);
```

```
        // VIEW OPERATIONS
```

```
        System.out.println("peek(): " + dq.peek());
```

```
        System.out.println("peekFirst(): " + dq.peekFirst());
```

```
        System.out.println("peekLast(): " + dq.peekLast());
```

```
        System.out.println("getFirst(): " + dq.getFirst());
```

```

System.out.println("getLast(): " + dq.getLast());
// STACK OPERATIONS
dq.push(100);
System.out.println("After push(): " + dq);
System.out.println("pop(): " + dq.pop());
// REMOVE OPERATIONS
System.out.println("remove(): " + dq.remove());
System.out.println("removeFirst(): " + dq.removeFirst());
System.out.println("removeLast(): " + dq.removeLast());
dq.remove(20);
System.out.println("After remove(20): " + dq);
System.out.println("poll(): " + dq.poll());
System.out.println("pollFirst(): " + dq.pollFirst());
System.out.println("pollLast(): " + dq.pollLast());
// UTILITY OPERATIONS
dq.add(50);
dq.add(60);
dq.add(70);
System.out.println("Deque: " + dq);
System.out.println("Contains 60? " + dq.contains(60));
System.out.println("Size: " + dq.size());
System.out.println("Is Empty? " + dq.isEmpty());
// ITERATOR
System.out.print("Iterator: ");
Iterator<Integer> it = dq.iterator();
while (it.hasNext()) {
    System.out.print(it.next() + " ");
}
System.out.println();
// TO ARRAY
Object arr[] = dq.toArray();
System.out.print("Array: ");
for (Object x : arr) {
    System.out.print(x + " ");
}

```

```

    }
    System.out.println();
    // CLEAR
    dq.clear();
    System.out.println("After clear(): " + dq);
    System.out.println("Is Empty Now? " + dq.isEmpty());
}
}

```

OUTPUT:

```

After Insertions: [1, 5, 10, 20, 30, 40]
peek(): 1
peekFirst(): 1
peekLast(): 40
getFirst(): 1
getLast(): 40
After push(): [100, 1, 5, 10, 20, 30, 40]
pop(): 100
remove(): 1
removeFirst(): 5
removeLast(): 40
After remove(20): [10, 30]
poll(): 10
pollFirst(): 30
pollLast(): null
Deque: [50, 60, 70]
Contains 60? true
Size: 3
Is Empty? false
Iterator: 50 60 70
Array: 50 60 70
After clear(): []
Is Empty Now? True

```

PRIORITY QUEUE: MIN HEAP:

OPERATIONS:


```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        PriorityQueue<Integer> pq =new PriorityQueue<>();

        // INSERT

        pq.add(30);

        pq.offer(10);

        pq.add(20);

        pq.offer(40);

        System.out.println("Min Heap: " + pq);

        // VIEW

        System.out.println("peek(): "+ pq.peek());

        System.out.println("element(): "+ pq.element());

        // REMOVE

        System.out.println("poll(): "+ pq.poll());

        System.out.println("remove(): "+ pq.remove());

        // Add again

        pq.add(50);

        pq.add(60);

        // Remove specific

        pq.remove(50);

        System.out.println("After remove(50): "+ pq);

        // UTILITY

        System.out.println("contains(60): "+ pq.contains(60));

        System.out.println("size(): "+ pq.size());

        System.out.println("isEmpty(): "+ pq.isEmpty());

        // ITERATOR

        System.out.print("Iterator: ");

        Iterator<Integer> it =pq.iterator();

        while(it.hasNext())

            System.out.print(it.next()+" ");

        System.out.println();

        // ARRAY

        Object arr[] =pq.toArray();
```

```

        System.out.print("Array: ");
        for(Object x: arr)
            System.out.print(x+" ");
        System.out.println();
        // CLEAR
        pq.clear();
        System.out.println( "After clear(): "+ pq);
    }
}

```

OUTPUT:

```

Min Heap: [10, 30, 20, 40]
peek(): 10
element(): 10
poll(): 10
remove(): 20
After remove(50): [30, 40, 60]
contains(60): true
size(): 3
isEmpty(): false
Iterator: 30 40 60
Array: 30 40 60
After clear(): []

```

PRIORITY ORDER: MAX HEAP

OPERATIONS:

```

import java.util.*;

public class Main {

    public static void main(String[] args) {

        PriorityQueue<Integer> pq =new PriorityQueue<>(Collections.reverseOrder());

        // INSERT

        pq.add(30);
        pq.offer(10);
        pq.add(20);
        pq.offer(40);

        System.out.println("Max Heap: " + pq);
    }
}

```

```

// VIEW
System.out.println("peek(): "+ pq.peek());
System.out.println("element(): "+ pq.element());
// REMOVE
System.out.println("poll(): "+ pq.poll());
System.out.println("remove(): "+ pq.remove());
// Add again
pq.add(50);
pq.add(60);
// Remove specific
pq.remove(50);
System.out.println("After remove(50): "+ pq);
// UTILITY
System.out.println("contains(60): "+ pq.contains(60));
System.out.println("size(): "+ pq.size());
System.out.println("isEmpty(): "+ pq.isEmpty());
// ITERATOR
System.out.print("Iterator: ");
Iterator<Integer> it =pq.iterator();
while(it.hasNext())
    System.out.print(it.next()+" ");
System.out.println();
// ARRAY
Object arr[] =pq.toArray();
System.out.print("Array: ");
for(Object x: arr)
    System.out.print(x+" ");
System.out.println();
// CLEAR
pq.clear();
System.out.println("After clear(): "+ pq);
}
}

```

OUTPUT:

Max Heap: [40, 30, 20, 10]

peek(): 40

element(): 40

poll(): 40

remove(): 30

After remove(50): [60, 10, 20]

contains(60): true

size(): 3

isEmpty(): false

Iterator: 60 10 20

Array: 60 10 20

After clear(): []

REVERSE THE FIRST K ELEMENTS OF A QUEUE USING A STACK AND DISPLAY THE UPDATED QUEUE

```
import java.util.*;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        ArrayDeque<Integer> q = new ArrayDeque<>();
```

```
        Stack<Integer> s=new Stack<>();
```

```
        // INSERT OPERATIONS
```

```
        q.add(10);
```

```
        q.add(30);
```

```
        q.add(20);
```

```
        q.add(50);
```

```
        q.add(40);
```

```
        System.out.println("After Insertions: " + q);
```

```
        int k=3;
```

```
        for(int i=0;i<k;i++){
```

```
            s.push(q.poll());
```

```
        }
```

```
        while(!s.isEmpty()){
```

```
            q.offer(s.pop());
```

```
        }
```

```
        for(int i=0;i<q.size()-k;i++){
```

```
            q.offer(q.poll());
```

```

    }

    System.out.println("Queue after reversing k no. of elements: "+q);

}

}

```

OUTPUT:

After Insertions: [10, 30, 20, 50, 40]

Queue after reversing k no. of elements: [20, 30, 10, 50, 40]

Using only queue(addFirst and addLast):

```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        int arr[] = new int[n];
        System.out.println("Enter elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        System.out.print("Enter k: ");
        int k = sc.nextInt();
        ArrayDeque<Integer> q = new ArrayDeque<>();
        for (int i = 0; i < k; i++) {
            q.addFirst(arr[i]);
        }
        // Remaining elements
        for (int i = k; i < arr.length; i++) {
            q.addLast(arr[i]);
        }
        System.out.println(
            "Queue after reversing k no. of elements: " + q);
    }
}

```

OUTPUT:

Enter number of elements:

5

Enter elements:

10

30

20

50

40

Enter k:

Queue after reversing k no. of elements: [30, 20, 10, 50, 40]

SET

Feature	HashSet	LinkedHashSet	TreeSet
Duplicates	✗	✗	✗
Order	No	Insertion	Sorted
Internal	Hash Table	Hash + Linked	Tree
Complexity	O(1)	O(1)	O(log n)

HASHSET

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        HashSet<Integer> hs =new HashSet<>();
        // INSERT
        hs.add(10);
        hs.add(20);
        hs.add(30);
        hs.add(10);
        System.out.println("HashSet: " + hs);
        // SEARCH
        System.out.println("Contains 20: " + hs.contains(20));
        // SIZE
        System.out.println("Size: " + hs.size());
        // REMOVE
        hs.remove(20);
        System.out.println("After remove: " + hs);
        // EMPTY
        System.out.println("isEmpty(): " + hs.isEmpty());
        // ITERATOR
        System.out.print("Iterator: ");
        Iterator<Integer> it =hs.iterator();
        while(it.hasNext())
            System.out.print(it.next()+" ");
        System.out.println();
        // ARRAY
        Object arr[] =hs.toArray();
        System.out.print("Array: ");
        for(Object x: arr)
            System.out.print(x+" ");
        System.out.println();
        // CLEAR
        hs.clear();
        System.out.println("After clear(): " + hs);
    }
}
```

OUTPUT:

```
HashSet: [20, 10, 30]
Contains 20: true
Size: 3
After remove: [10, 30]
isEmpty(): false
Iterator: 10 30
Array: 10 30
After clear(): []
```

LINKEDHASHSET

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        LinkedHashSet<Integer> hs =new LinkedHashSet<>();

        // INSERT

        hs.add(10);

        hs.add(20);

        hs.add(30);

        hs.add(10);

        System.out.println("LinkedHashSet: " + hs);

        // SEARCH

        System.out.println("Contains 20: "+ hs.contains(20));

        // SIZE

        System.out.println("Size: "+ hs.size());

        // REMOVE

        hs.remove(20);

        System.out.println("After remove: "+ hs);

        // EMPTY

        System.out.println("isEmpty(): "+ hs.isEmpty());

        // ITERATOR

        System.out.print("Iterator: ");

        Iterator<Integer> it =hs.iterator();

        while (it.hasNext())

            System.out.print(it.next() + " ");

        System.out.println();

        // ARRAY

        Object arr[] =hs.toArray();

        System.out.print("Array: ");
```

```

    for (Object x : arr)
        System.out.print(x + " ");
    System.out.println();
    // CLEAR
    hs.clear();
    System.out.println("After clear(): "+ hs);
}
}

```

OUTPUT:

LinkedHashSet: [10, 20, 30]

Contains 20: true

Size: 3

After remove: [10, 30]

isEmpty(): false

Iterator: 10 30

Array: 10 30

After clear(): []

TREESET

```

import java.util.*;

public class Main {

    public static void main(String[] args) {
        TreeSet<Integer> ts =new TreeSet<>();
        // INSERT
        ts.add(30);
        ts.add(10);
        ts.add(20);
        ts.add(40);
        ts.add(10);
        System.out.println("TreeSet: " + ts);
        // SEARCH
        System.out.println("Contains 20: "+ ts.contains(20));
        // FIRST / LAST
        System.out.println("first(): "+ ts.first());
        System.out.println("last(): "+ ts.last());
    }
}

```



```

// NAVIGATION
System.out.println("higher(20): "+ ts.higher(20));
System.out.println("lower(20): "+ ts.lower(20));
System.out.println("ceiling(25): "+ ts.ceiling(25));
System.out.println("floor(25): "+ ts.floor(25));

// REMOVE
ts.remove(20);

System.out.println("After remove(): "+ ts);

// SIZE
System.out.println("Size: "+ ts.size());

// EMPTY
System.out.println("isEmpty(): "+ ts.isEmpty());

// ITERATOR
System.out.print("Iterator: ");
Iterator<Integer> it =ts.iterator();
while (it.hasNext()) {
    System.out.print(it.next() + " ");
}

System.out.println();

// CLEAR
ts.clear();

System.out.println("After clear(): "+ ts);
}
}

```

OUTPUT:

TreeSet: [10, 20, 30, 40]

Contains 20: true

first(): 10

last(): 40

higher(20): 30

lower(20): 10

ceiling(25): 30

floor(25): 20

After remove(): [10, 30, 40]

Size: 3

isEmpty(): false

Iterator: 10 30 40

After clear(): []